

Contracts and Schemas

Zod schemas that define and enforce the shape of data exchanged between pipeline stages at runtime. Every major boundary in the documentation-generation pipeline validates its inputs and outputs against these contracts, providing a single source of truth for data structures.

Purpose

- **Runtime validation** – Each schema uses [Zod](#) to parse and validate objects at stage boundaries, catching malformed data early.
- **Shared vocabulary** – Downstream modules import schemas rather than inventing ad-hoc types, keeping the pipeline consistent.
- **Type inference** – TypeScript types are inferred from the schemas (`z.infer<...>`), so runtime checks and compile-time types stay in sync.

Contract Files

File	Key Exports
<code>liminal-docgen/src/contracts/analysis.ts</code>	<code>rawFileTreeNodeSchema</code> , <code>rawNodeSchema</code> , <code>rawCallRelationshipSchema</code> , <code>rawAnal</code>
<code>liminal-docgen/src/contracts/configuration.ts</code>	<code>configurationRequestSchema</code> , <code>configurationFileSchema</code> , <code>resolvedConfiguratio</code>
<code>liminal-docgen/src/contracts/generation.ts</code>	<code>moduleGenerationResultSchema</code> , <code>overviewGenerationResultSchema</code>
<code>liminal-docgen/src/contracts/metadata.ts</code>	<code>generatedDocumentationMetadataSchema</code> , <code>metadataWriteRequestSchema</code>
<code>liminal-docgen/src/contracts/planning.ts</code>	<code>modulePlanSchema</code> , <code>ModulePlan</code> type, <code>CLUSTERING_THRESHOLD</code>
<code>liminal-docgen/src/contracts/publish.ts</code>	<code>PublishRequestSchema</code> , <code>PublishResultSchema</code>
<code>liminal-docgen/src/contracts/quality-review.ts</code>	<code>reviewFilePatchSchema</code> , <code>reviewPatchPayloadSchema</code>
<code>liminal-docgen/src/contracts/validation.ts</code>	<code>validationRequestSchema</code> , <code>validationFindingSchema</code> , <code>validationResultSchema</code>

Dependencies

- **Type Definitions** – `contracts/validation.ts` imports shared types from `types/index.ts`. The planning contract also re-exports its `ModulePlan` type, which the Type Definitions module consumes.

Design Notes

- Each contract file is intentionally small (8–72 LOC) and focused on a single pipeline concern.
- The `CLUSTERING_THRESHOLD` constant in `planning.ts` is co-located with the planning schema because it directly governs how module plans are validated.
- No contract file contains business logic; they export only schemas, inferred types, and related constants.